STATE / EVIDENCE / DECISION / PROOF

Authentication & Session Security Explained

Understand account trust, prove the weakness, and protect the full identity lifecycle

VERIFIED EVIDENCE

What was actually proved?

CURRENT AUTHORITY

What may happen now?

Follow the account from first proof to final logout.

Find the weakness. Prove the effect. Verify the repair.

Practice with real local account and token workflows.

Haroon Shahid

First edition / Version 1.0 / 1 November 2026

Before you begin

Authentication & Session Security Explained

Understand account trust, prove the weakness, and protect the full identity lifecycle

By Haroon Shahid. First edition, version 1.0. 1 November 2026. Prepared with AI assistance.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under LICENSE.txt. The separately identified lab code is MIT licensed. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and accounts you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. The local Identity Harbor lab uses real standard-library password hashing, TOTP, random grants, HMAC-signed JWTs, session transitions and fixed-loopback HTTP with fictional accounts. Fixtures and clock controls are privileged teaching tools. WebAuthn, external identity providers, real mail, framework apps and distributed systems receive review guidance; they were not implemented by this lab. It is not a production authentication library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format. On narrow screens, swipe sideways within wide diagrams and tables. No full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package includes all 64 chapters, answered reviews, the offline reading edition, practice workbooks, reference sheets, and the local Identity Harbor lab.

Contents

Before you begin 2
Contents 3
Introduction 4
Learning roadmap 5
01. Follow an account, not only a login form 7
02. Understand the browser and server conversation 9
03. Cookies carry values; sessions give them meaning 11
Sources and further reading 13

Introduction

Mira signs in to HarborDesk, a fictional service. The password is right. The page asks for a second factor. Mira has not entered it yet.

Should a separate account endpoint return Mira's private note now?

That small question opens the whole subject. A password result, a complete login, a session, a recovery message and permission to perform an action are different facts. Confusing these facts creates unsafe decisions.

We follow one account through login, factors, recovery, changes, token renewal and logout. Start with requests and cookies. Use that foundation later to understand JWTs, passkeys and distributed revocation.

The four questions we keep asking

1. **State:** Where is this account or session in the journey?
2. **Evidence:** What was actually proved, by whom, and when?
3. **Decision:** Which exact action may the server permit now?
4. **Proof:** Which observation shows that the boundary held or failed?

Think of an airport. A booking, a checked identity, a boarding pass and permission to enter the cockpit are not interchangeable. Web evidence can also be copied, replayed and processed concurrently. We examine those limits directly.

What you will be able to do

You will map account state, design paired tests, prove a bounded failure and verify its repair. A surprising error is a lead. A protected operation accepted under insufficient evidence is stronger proof. Learn to state both the result and its limits.

The optional Identity Harbor lab contains deliberately weak workflows beside stricter ones. It performs real password hashing, HOTP/TOTP, random-token handling, HMAC signing, expiry checks and local HTTP. Effects stay inside two fictional accounts and memory. The lab's fixtures provide privileged teaching visibility. A fixture that hands you a code or signs a negative test token is not an attacker capability in a real service.

Passkeys, real email delivery, OAuth providers, framework integrations and distributed systems receive detailed review guidance. They are not secretly implemented by the local lab. The validation report names what was executed. Keeping that boundary clear is part of professional learning.

How to use the kit

Read a small section. Explain the diagram aloud. Predict an outcome before revealing its answer. Try one mission. Record a fact, an inference and a limit. Return after a day and explain the same boundary without looking. Use the mastery tracker to choose your next piece of evidence, not to award yourself a title.

Use authorized systems, dummy data and the smallest reversible proof. This self-study book supports independent work; it does not certify expertise or guarantee discoveries.

Start with the account journey. Everything that follows connects back to it.

Learning roadmap

Choose a starting route

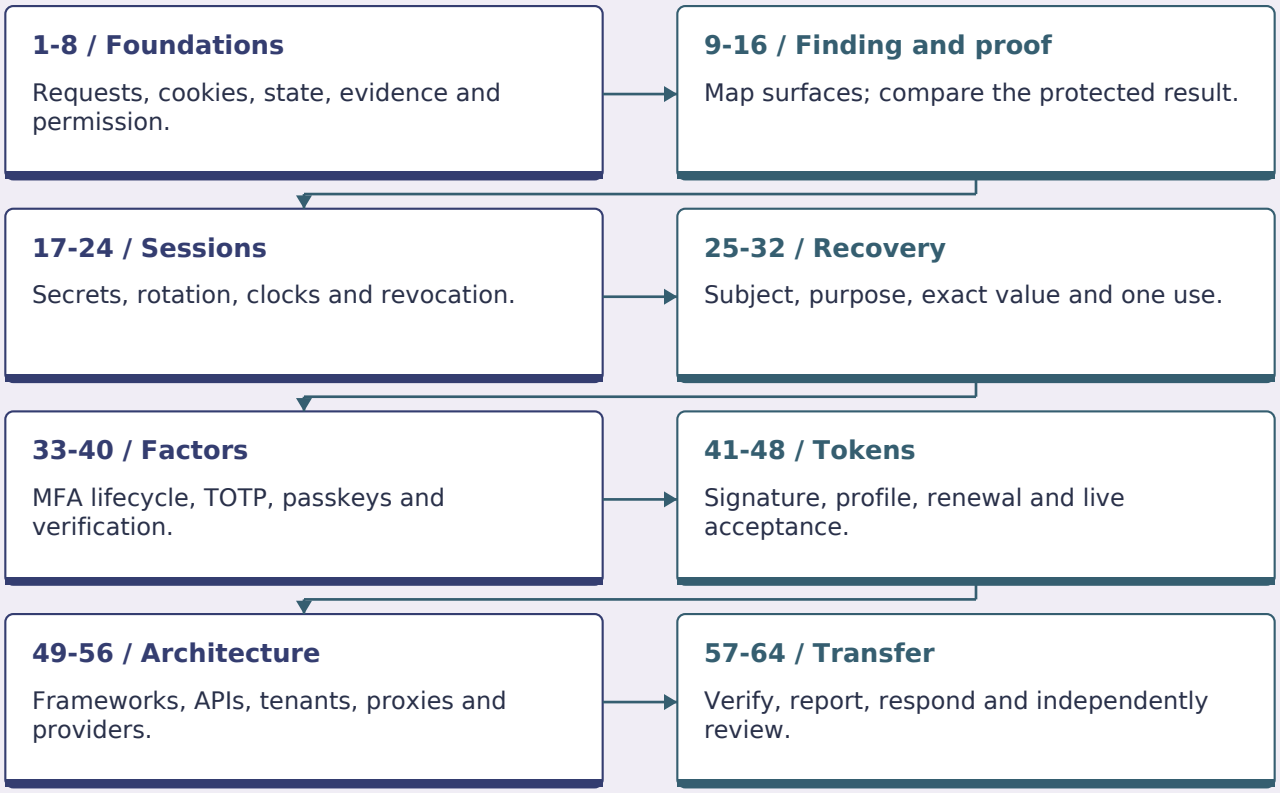
- New learner: Chapters 1-8, then the first four missions. Explain requests, cookies, state and protected actions.

- Developer: foundations, then Chapters 17-48. Write a contract for every credential and transition.

- Authorized tester: Chapters 8-16, then lifecycle sections. Produce paired tests and a clean evidence trail.

- Architect: Chapters 49-56 after the evidence model. Map enforcement, keys, tenants and revocation.

Learning dependencies / identity, evidence and current authority



Read left to right within each stage, then follow the connector to the next row. Each later stage uses the earlier mental model.

Eight stages, one connected model

Stage	Chapters	Observable milestone
Foundations	1-8	Separate identity, evidence, session and permission
Finding and proof	9-16	Prove a partial-session gate failure with one dummy read
Session lifecycle	17-24	Verify rotation, three clocks, current logout and all-session revocation
Recovery and change	25-32	Bind a one-use grant to subject, purpose, time and exact value
MFA and passkeys	33-40	Explain factor independence, replay resistance and WebAuthn verification
Tokens and renewal	41-48	Separate cryptographic validation from live resource acceptance
Real architecture	49-56	Review framework, API, tenant, proxy and identity-provider boundaries
Transfer	57-64	Produce and defend an independent report with regression evidence

Practice order

Complete missions 1-10 after Chapters 1-18. Complete 11-17 as you study attempts and sessions. Continue with 18-25 for recovery and changes, then 26-35 for signed tokens, renewal, alternate sign-in evidence and request intent. Mission 36 is the independent capstone. The numbered mission instructions give their own prerequisites; you may revisit a later mission when its chapter becomes familiar.

Use the 48 predictions for retrieval practice and the 28 review cases for transfer beyond the lab. Every chapter has three checks with explained answers. Its second figure is a compact state/evidence/decision/proof revision sheet. The separate visual atlas collects all 64 of those sheets.

Progress without rushing

On day one, draw the account journey. On your next study day, redraw it from memory and add one failed boundary. After a week, explain a case you have not practiced. After a month, use the capstone rubric on a fresh synthetic design. Time spent is not a pass mark. A correct explanation supported by repeatable evidence is more useful.

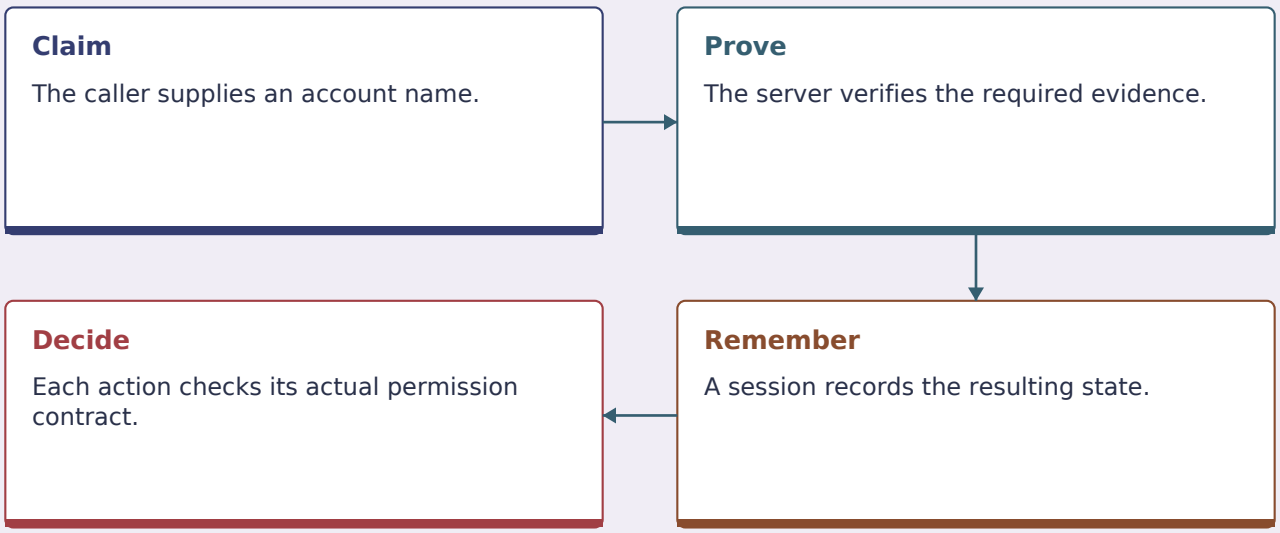
I. UNDERSTAND IDENTITY AND TRUST

01. Follow an account, not only a login form

Learn to: Separate claimed identity, verified evidence, session state and permission for the next action.

Think of it this way: A visitor can state a name, show a badge and enter a lobby without receiving permission to open the records room.

01A / An account moves through different trust states



Follow evidence into state and state into permission; a cookie alone proves neither.

Meet HarborDesk

Mira and Ben use HarborDesk, a fictional team service. Mira belongs to the Blue company. Ben belongs to Amber. Both can sign in, read a private note and change their contact details. A sensitive approval requires recent proof of identity. The same account appears in several workflows, but each workflow makes a different promise.

Authentication means checking evidence about an identity. An identity is the account or person being claimed. A password, a one-time code or a public-key assertion can provide evidence. The server must decide which account that evidence belongs to and what it proves.

Authorization means deciding whether this actor may perform this action on this resource. A correct login does not let Mira read Ben's private note. An **actor** is the user or component making the request. A **resource** is the thing being accessed, such as an account record or approval.

A session remembers a decision

HTTP requests arrive separately. A **session** connects later requests with an earlier authentication event. A session identifier is a secret handle the client presents so the server can find that state. Later chapters compare server-stored sessions with signed tokens.

A session can exist before full authentication. HarborDesk may remember that a password was correct while waiting for MFA, multi-factor authentication. If a protected route checks only that a session exists, it can grant access too early. The session's state matters more than its mere presence.

Follow the whole journey

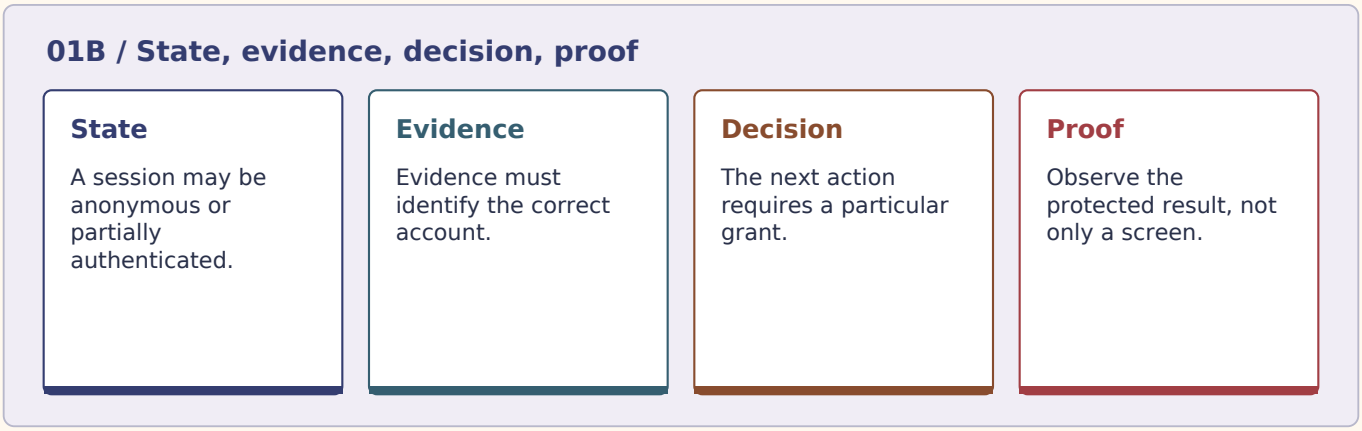
Registration creates an account record. Verification proves a particular contact channel. Login establishes authentication evidence. Recovery offers another route to regain access. Session management preserves or ends the resulting authority. Email changes, new factors and remembered devices can change future access.

These are connected decisions, not unrelated forms. A strong password check can coexist with a recovery flow that changes the wrong account. A good MFA screen can coexist with an API that accepts the password-only stage.

Your first prediction

Mira enters her correct password and receives a session cookie. She has not entered an OTP. Should the server return her MFA-protected note? Under HarborDesk's contract, no. A weak gate might still return it. That precise mismatch will become your first local finding.

Keep asking: which account, what evidence, which state, which action and what observable result? This question stays useful from the first chapter through advanced token and recovery analysis.



Key takeaway. Follow evidence into state and state into permission; a cookie alone proves neither.

Check your understanding

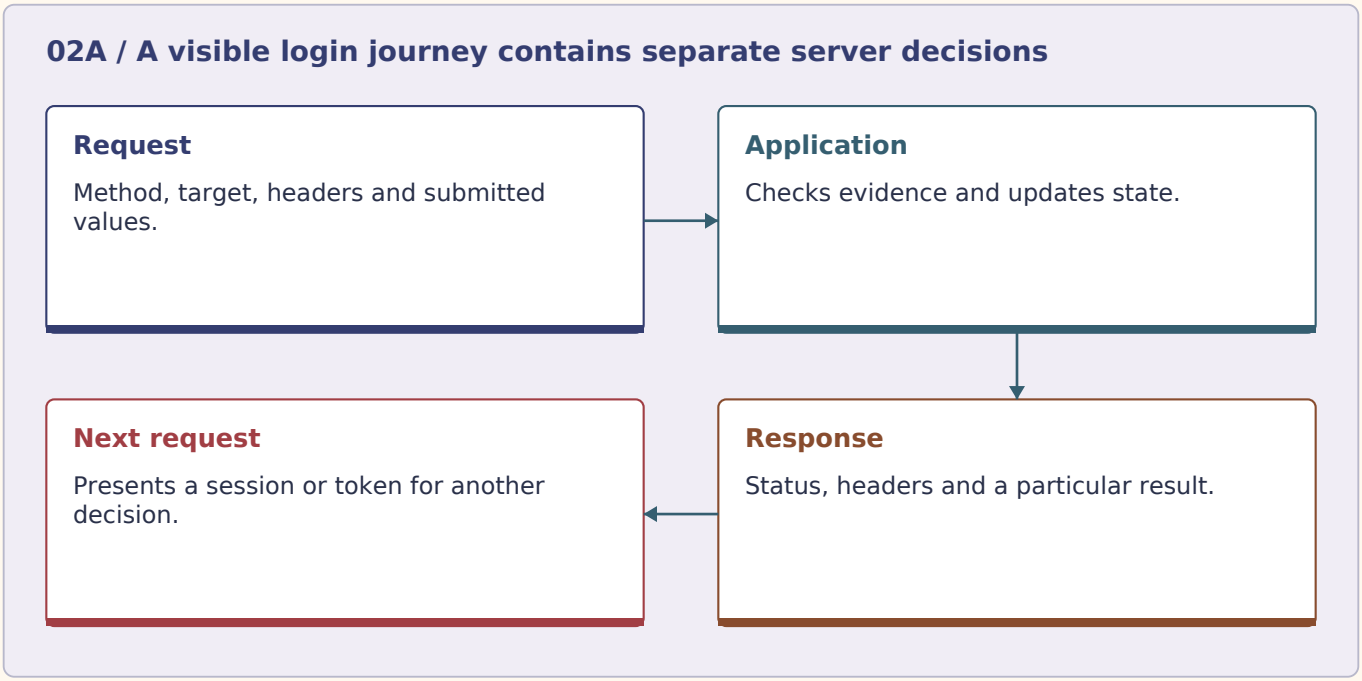
1.1 Does receiving a cookie prove full login?	**1.2** How do authentication and authorization differ?	**1.3** Why follow recovery too?

Answer explanations are in the full ebook. Sources: AUTH, AUTHZ, PORTAUTH

02. Understand the browser and server conversation

Learn to: Read URLs, HTTP requests, responses and the reason web applications need state.

Think of it this way: A help desk receives separate messages; it needs a reliable case reference to connect them.



Read the real operation and its result instead of treating UI navigation as access control.

Two programs exchange messages

A browser is a program that requests and displays web content. A server handles those requests. The application is the feature logic running on the server or related services. A database stores records. The server and database are roles, not necessarily different physical computers.

A URL is a resource address. In http://127.0.0.1:8960/account, http is the scheme, 127.0.0.1 is the host, 8960 is the port selecting a listener, and /account is the path. A domain is a readable name; DNS resolves names to network addresses. Loopback addresses refer to your own computer.

An HTTP request includes a method, target, headers and sometimes a body. A response includes a status, headers and sometimes a body. POST often submits a login. GET commonly retrieves a page. Neither method automatically establishes permission.

```
POST /login HTTP/1.1
Host: app.example.test
Content-Type: application/json

{"user":"mira","password":"fictional-example"}
```

This is a message sketch, not a complete replay request. A real client supplies correct framing and length. The server should not trust a body field merely because it came from the normal interface.

State must be designed

HTTP does not automatically remember a user's authentication across separate requests. The application introduces continuity using a session secret or another credential. A later request may carry a cookie or an Authorization header. What the server accepts depends on its authentication and authorization policy.

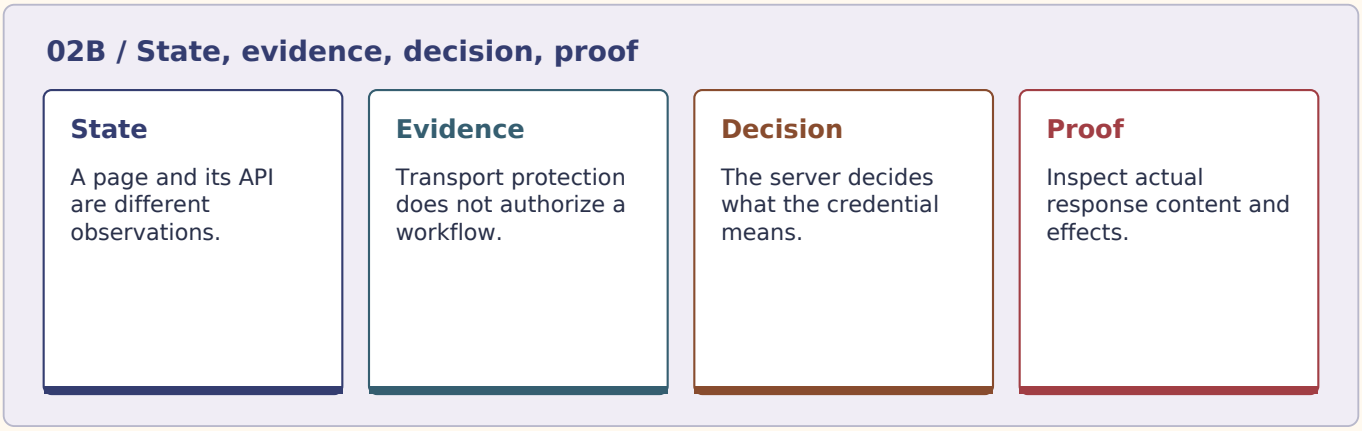
The browser may show an MFA page while an API endpoint already returns private data. A frontend redirect describes user experience. Server-side state and enforcement determine actual access. Inspect the request that performs the protected operation.

HTTPS has a specific job

HTTPS uses TLS to protect the connection under its authentication and encryption rules. It does not stop the application from accepting the wrong recovery token or skipping MFA. A phishing site can have HTTPS for its own domain. Check the intended origin and workflow as well as transport.

Follow one real operation

Open the browser Network panel. Perform a normal login and protected read. Record method, path, necessary fields, response and account state. A 200 containing a login page is not the same as a 200 containing the private note. Observe the response meaning and any actual side effect.



Key takeaway. Read the real operation and its result instead of treating UI navigation as access control.

Check your understanding

2.1 Is a database the same thing as a server?	**2.2** Does HTTPS prevent authentication logic bugs?	**2.3** Does a 200 status prove private access?

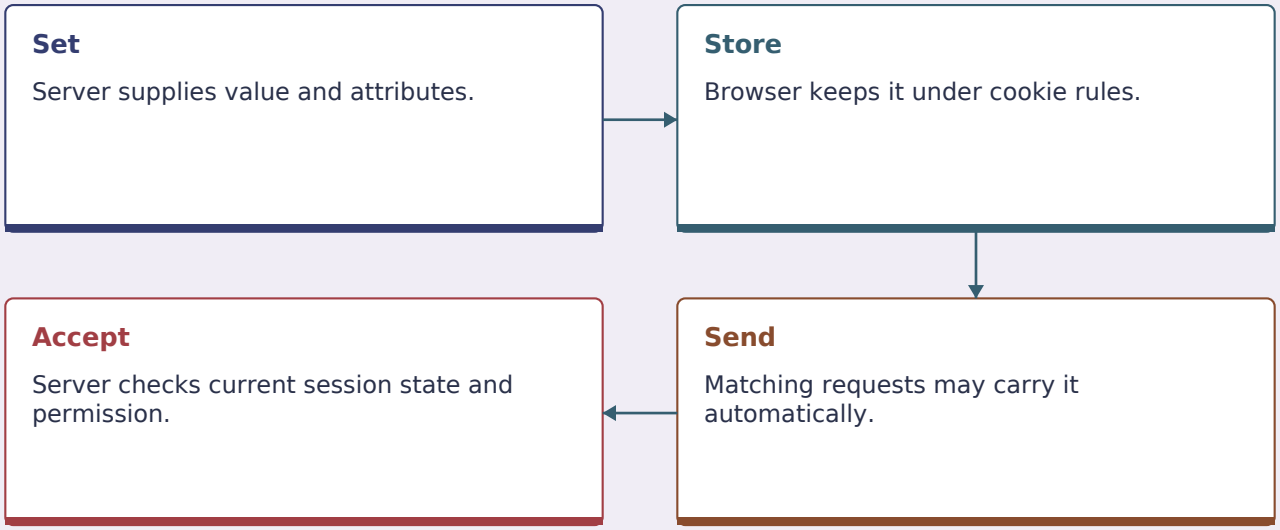
Answer explanations are in the full ebook. Sources: HTTP, TLS, PORTAUTH

03. Cookies carry values; sessions give them meaning

Learn to: Explain cookie attributes, scope and automatic browser sending.

Think of it this way: A claim ticket is a small label. The warehouse record determines what the ticket may collect.

03A / A cookie and its server record have separate lifecycles



Cookie controls support a session policy; they do not replace current server-side acceptance rules.

A cookie is a browser-stored value

A response can use Set-Cookie to ask a browser to store a name/value pair with attributes. The browser later sends matching cookies in a Cookie request header. A session cookie often carries an opaque identifier rather than the account's complete data.

```
Set-Cookie: session=opaque-random-value; Path=/; Secure; HttpOnly; SameSite=Lax
```

This is an illustrative production-style cookie. It does not create a complete secure session design. The server still needs unpredictable identifiers, proper storage, state checks, expiry and invalidation.

Attributes answer different questions

Secure restricts sending to secure transport under browser rules. HttpOnly prevents ordinary page JavaScript from directly reading that cookie value. SameSite restricts some cross-site sending; Strict, Lax and None have different behavior. None requires Secure in modern browsers. Do not treat SameSite as complete CSRF protection.

Domain affects host scope. Omitting Domain creates a host-only cookie. Path affects when a cookie is sent; it is not a general security isolation boundary. Cookie scope does not include ports. A different port can change browser origin without separating cookies for the same host.

An origin is generally scheme, host and effective port. A site is a different grouping used by mechanisms such as SameSite. Two sibling subdomains can be cross-origin while still same-site. This matters when another application on the site is less trusted.

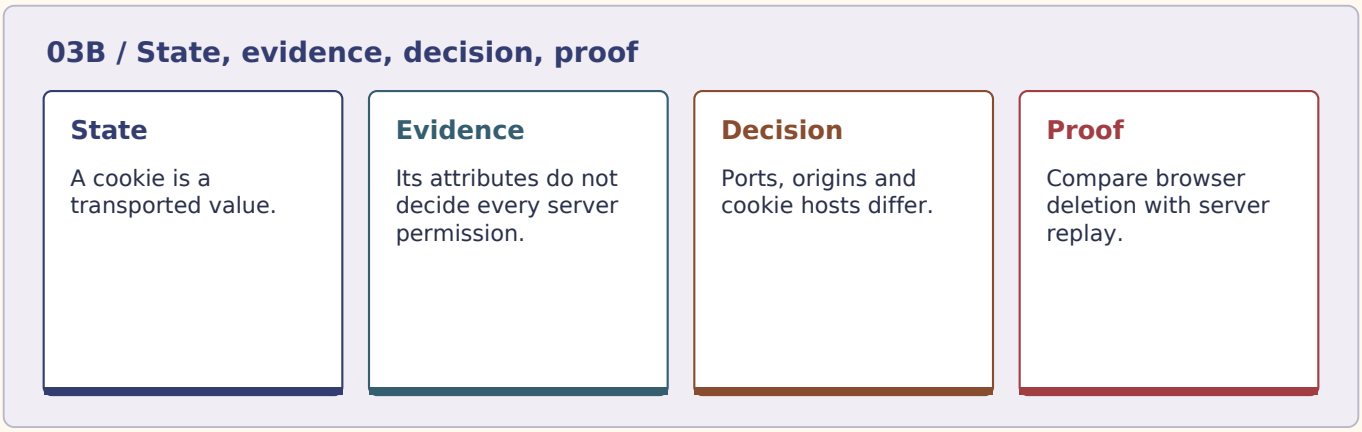
Cookie lifetime and server lifetime differ

A persistent cookie may have Max-Age or Expires. A session cookie has no explicit persistence expiry, but browser restoration can preserve session state. Closing a tab or browser is not reliable server logout. The server needs its own idle, absolute and revocation checks.

Deleting the browser cookie only changes that browser's stored copy. Another captured copy may still be accepted if the server record remains valid. You will test this distinction using a private local replay slot.

The lab's deliberate exception

Identity Harbor uses host-only, HttpOnly, SameSite=Strict cookies on local HTTP. It omits Secure for that documented local setup. This is not a production cookie recommendation. The guide distinguishes local convenience from actual HTTPS deployment and recommends verifying the complete cookie and server contract.



Key takeaway. Cookie controls support a session policy; they do not replace current server-side acceptance rules.

Check your understanding

3.1 Does HttpOnly stop every authenticated action from XSS?	**3.2** Do different ports isolate cookies?	**3.3** Does closing a tab invalidate the server session?

Answer explanations are in the full ebook. Sources: COOKIE, ORIGIN, SESSION, CSRF

Sources and further reading

Research cutoff: 1 November 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

AUTH - OWASP: Authentication

Consulted targeted sections. Identity checks, sensitive changes and account lifecycle. No single setting implements the complete product contract.

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

SESSION - OWASP: Session Management

Consulted targeted sections. Session identifiers, rotation, expiry and invalidation. Browser cookie lifetime and server acceptance are different.

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

HTTP - RFC 9110: HTTP Semantics

Targeted further reading. Requests, responses, methods and representation meaning.

https://www.rfc-editor.org/rfc/rfc9110.html

COOKIE - MDN: Set-Cookie

Consulted targeted sections. Host/path scope, Secure, HttpOnly, SameSite and prefixes; a different port does not isolate cookies.

https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie

ORIGIN - WHATWG HTML: Origins

Targeted further reading. Origin identity and isolation, distinct from sites and cookie scope.

https://html.spec.whatwg.org/multipage/browsers.html#origins

CSRF - OWASP: CSRF Prevention

Targeted further reading. Intent checks for ambient browser authority, including sensitive account actions.

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

AUTHZ - OWASP: Authorization

Targeted further reading. Permission decisions remain necessary after identity verification.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

PORTAUTH - PortSwigger: Authentication vulnerabilities

Consulted targeted sections. Primary educational taxonomy and controlled lab methodology.

https://portswigger.net/web-security/authentication

TLS - OWASP: Transport Layer Security

Targeted further reading. Transport protection is distinct from credential, workflow and authorization correctness.

https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html